

BPO Workforce Operations Analytics

Unified product definition, technical implementation plan, and AI delivery model

Version 1.0 | 12 September 2026

Build an internal system that helps managers explain scheduled capacity, connect it to authoritative work output, and resolve operational exceptions using trustworthy evidence.

The first release uses a Windows collector, a management web interface, and PostgreSQL on suitable company infrastructure. The plan prioritizes accurate definitions, minimal collection, bounded operating cost, and a measured pilot.

Decisions at a glance

Decision	First release baseline
Deployment	One company, existing infrastructure where suitable
Core stack	C# Windows collector, ASP.NET Core, React, PostgreSQL
Additional services	No ClickHouse or paid runtime AI dependency
Pilot ladder	25 to 50 employees, then 200 after evidence gates
Measurement	Schedules, activity evidence, approved context, and imported output
Delivery control	AI lead, bounded specialist work, independent review

Planning status

The three original sketches have been consolidated and reviewed. Company-specific assumptions and acceptance targets are explicit. This document is an implementation baseline; it does not claim that software has been built, tested, or deployed.

Contents

Project Definition	3
1 Purpose and project decision	3
2 Context and assumptions	3
3 Objectives and business outcomes	3
4 Users and access	4
5 Core workflows	4
6 First release scope	5
7 Metric contract	5
8 Quality and acceptance targets	6
9 Collection and trust policy	7
10 Decision ownership and boundaries	8
Technical Implementation Plan	9
1 Architecture decision	9
2 Deployment and identity	9
3 Windows collection design	10
4 Local durability and transfer	10
5 Database model	11
6 Aggregation and metric implementation	12
7 API and interface contracts	13
8 Capacity model and cost controls	13
9 Operations and release safety	14
10 Delivery work packages and evidence gates	14
11 Required verification matrix	15
12 Technical references	15
AI Team Delivery and Project Control	16
1 Operating model	16
2 Roles and assignment rules	16
3 Required agent report	16
4 Initial implementation queue	17
5 Cost and change control	17
6 Risk register and ownership	18
7 Reporting and company dependencies	19

PART 1

Project Definition

Version 1.0 | 12 September 2026 | Project lead: coordinating AI agent

1 Purpose and project decision

Build an internal operations analytics system that helps the COO and team leaders understand how scheduled capacity is used, identify data and process problems, and follow up on operational exceptions. The first release will combine workforce schedules, conservative desktop activity summaries, and imported operational results. Every reported measure must identify its source, coverage, definition, and limitations.

The company will operate one deployment. PostgreSQL will serve both operational records and initial analytics. Existing company compute, identity, network, and backup facilities are preferred when they meet the requirements. The initial release has no mandatory ClickHouse, commercial analytics, hosted queue, or paid AI inference dependency.

The system succeeds when managers can resolve useful operational questions with trustworthy evidence. Installing a tracker or producing a dashboard is not sufficient evidence of business value.

2 Context and assumptions

The three original PDFs are discovery inputs. This document and the technical plan supersede their implementation recommendations where they differ. The original documents remain unchanged. Competitor descriptions and predicted commercial returns in those notes are not treated as verified requirements.

Planning assumptions, rather than confirmed company facts, are:

- One company with several teams and client accounts; internal deployment only.
- A Windows workstation pilot of 25 to 50 people, followed by a controlled expansion to 200. A 1,000-agent synthetic workload is a capacity test, not a promised production limit.
- A company-managed Windows test device and a small Linux VM can be made available. Shared desktops, remote sessions, VPN access, and identity-provider compatibility require an early environment check.
- Initial roster, schedule, and work-output data can be supplied as approved CSV exports. Live CRM integrations are not required to begin development.
- Managers need daily and intraday visibility; a five-minute healthy-data freshness target is adequate for the pilot.
- Existing company authentication and deployment policies take precedence over development defaults.

The lead records new evidence in the decision log and adjusts the plan. Unknowns do not block synthetic-data development. Actual employee collection requires company-authorized deployment, a named operational owner, and an agreed collection policy. No geography, employment policy, regulatory certification, or permission to deploy to employees is inferred from this plan.

3 Objectives and business outcomes

Objective O1 is to explain scheduled capacity: distinguish observed activity, observed inactivity, approved non-desktop work, and missing or conflicting evidence.

Objective O2 is to reduce manual reporting effort: combine approved roster, schedule, and output exports into reproducible team reports, without repeated spreadsheet reconciliation.

Objective O3 is to make exceptions actionable: let a team leader investigate an unusual interval, record context, assign an action, and track its resolution.

Objective O4 is to control operating cost and reliability: use a small deployable system, bounded collection and retention, and tested recovery procedures.

Before the pilot, record two weeks of the team's current reporting effort, exception-resolution time, and existing output and quality measures. Proposed pilot value targets are a 30 percent reduction in report-preparation time and at least three verified, actionable process findings over two weeks. These are evaluation targets, not promised savings. Record the actual baseline, sample size, and result; do not declare success from activity changes alone.

Recovered revenue or billable hours will not be claimed unless a client-approved billing rule and authoritative billing data support the calculation. The initial release does not produce payroll or invoices.

4 Users and access

- COO and operations manager: view authorized teams, coverage, trends, imported output and quality, and unresolved exceptions. Cross-team comparisons show comparable work types and definitions.
- Team leader: inspect assigned employees and periods, validate schedule mismatches, document non-desktop work, and manage follow-up actions. Access follows effective-dated team assignment.
- Employee: see their own collected categories, collection status, daily explanation, and correction requests. The tray application visibly indicates collection and connectivity status.
- Platform administrator: manage enrollment, policies, imports, and health. Administration does not automatically grant employee-report access.
- Operational data owner: validate import mappings, source totals, work-type definitions, and report revisions. This may be an existing manager rather than a new job.

Access is enforced on the server for dashboards, employee details, exports, and background report jobs. A client account is a reporting and authorization boundary within the company, not a separate SaaS tenant. Employee IDs come from a stable company identifier; display names and email addresses are not matching keys.

5 Core workflows

Daily team review

A team leader selects a date and team. The dashboard first shows source freshness and scheduled-time coverage. It then shows time categories and available output and quality measures. The leader opens an exception, reviews its source intervals and schedule, records an explanation, and assigns a follow-up. A missing desktop interval is investigated as missing evidence, not automatically labeled an employee performance issue.

Operational trend review

An operations manager compares the same work type over time and reviews capacity alongside completed work and quality. The report discloses absent integrations and missing denominators. A role-specific application classification may support investigation but cannot independently establish productive output.

Import and reconciliation

An authorized user uploads a roster, schedule, or operational export, previews mapping and errors, and commits a validated batch. The system stores source identity, checksum, counts, rejected rows, reporting timezone, and revision. Repeated uploads cannot silently duplicate facts. Corrections create a traceable replacement version.

Employee correction

An employee identifies a period that needs context, such as approved training or a customer call. A scoped manager reviews the request. The original observation is retained and an approved annotation is added with author and reason. Reports distinguish observed data from business interpretation.

Device recovery

The agent buffers summaries during disconnection. Reconnection uploads acknowledged batches without duplicating time. If buffering overflows or collection stops, the dashboard shows a gap and the device-health view records the cause. A stopped process is a health event requiring investigation, not proof of misconduct.

6 First release scope

P0 requirements must pass before the employee pilot:

- R01: company identity integration, scoped roles, employee and device enrollment, revocation, and effective-dated team and client assignment.
- R02: validated roster and schedule CSV imports, including overnight shifts, timezone handling, breaks, and rejected-row reports.
- R03: visible Windows session collector for process identity, coarse active or inactive state, lock state, and collection health, with encrypted local buffering.
- R04: retry-safe ingestion, durable acknowledgment, clear missing-data indicators, and reproducible versioned aggregation.
- R05: daily team overview and employee timeline with drill-down, metric definitions, coverage, and freshness.
- R06: one agreed operational-output import mapping, supporting completed work and quality where the source contains them. Without it, the pilot is explicitly an activity-visibility pilot and cannot pass the output-linkage business gate.
- R07: role-specific application classification with effective dates; unknown applications remain unclassified. No default assumption that all browser use is productive or wasteful.
- R08: exception records, contextual annotations, correction requests, and audited state changes.
- R09: authorization-safe CSV export with formula-injection protection, report revision, filters, and metric definitions.
- R10: collection notice, retention configuration, access audit, device health, backup restore, and staged update and rollback procedures.

Deferred scope includes macOS, browser extensions and URL inspection, live CRM or telephony connectors, automated forecasting, staffing optimization, client billing, payroll, multi-company SaaS, screenshots, keystroke content or counts, clipboard capture, AI-generated employee scores, attrition prediction, and generative coaching. New scope must replace or defer existing work unless the lead explicitly revises capacity and cost assumptions.

7 Metric contract

All durations use seconds internally and a half-open interval convention: start inclusive and end exclusive. Reports state their timezone. Zero and unavailable are different values. Rates with a missing or zero denominator display unavailable with a reason.

Time reconciliation

Scheduled eligible time is imported scheduled time minus approved breaks and excluded absence. Preserve both the original schedule and its revision. Within eligible time, assign each instant exactly one category in this precedence order: approved non-desktop work; conflicting device evidence; observed locked or inactive; observed active; unknown. Inactivity is evaluated only when the collector is healthy. For a single session, locked state overrides its input state. Conflicts are resolved only by a recorded source-selection rule or approved correction.

The five categories sum to eligible time. Observations outside schedule are reported separately and are not automatically classified as overtime or billable work. For overlapping shifts, reject or explicitly normalize schedule conflicts before calculating a denominator. Authorized annotations do not alter raw observations.

Desktop telemetry coverage is the union of valid observed desktop intervals within eligible time divided by eligible time. Approved non-desktop annotations do not artificially improve telemetry coverage. Time explained by annotations is reported as a separate quantity. Show conflict duration separately even if desktop telemetry is present.

Observed active share is observed active desktop seconds divided by active plus locked or inactive desktop seconds, after removing annotated and conflicting periods. It describes input activity, not output or effort. Reports show coverage alongside it; comparison eligibility requires at least 95 percent valid coverage within expected desktop collection windows and no unresolved conflicts. Expected desktop windows exclude prospectively scheduled approved non-desktop periods; a later annotation cannot shrink this denominator. Continue displaying unadjusted telemetry coverage over eligible time alongside the eligibility measure.

Application-classified active share is active time in a configured application category divided by total eligible observed active time. Show unclassified active time explicitly. Rules are scoped to role or process and versioned. This is a usage measure, not a general productivity score.

Work and quality

Completed work is the deduplicated count of source-defined completed work items in a stated period and work type. Quality pass rate is passed reviewed items divided by reviewed items, with the sample count and source definition visible.

Output per approved work hour uses completed comparable items divided by authoritative approved work hours from an agreed source, stored with its own source revision. An optional approved-hours CSV feed supplies this denominator; until available the rate remains unavailable. Do not substitute keyboard-active hours. Average handling time is published only when the source supplies actual handling durations and matching completed items, with explicit rules for transfers, reopened work, and parallel work. Otherwise it remains unavailable.

Schedule variance in this release means a difference between imported expected presence and observed or approved evidence. It is not formal adherence unless an authoritative work-state integration supplies the necessary states. Idle duration is never automatically translated into financial loss.

8 Quality and acceptance targets

These targets are proposed release gates and must be measured, not assumed:

- **Correctness:** replaying the same accepted batch ten times leaves totals unchanged; late valid events produce one traceable report revision; approved categories reconcile exactly to eligible seconds.
- **Coverage:** at least 98 percent of scheduled eligible minutes have valid desktop evidence on healthy, supported pilot devices, excluding explicitly scheduled non-desktop periods from this gate's test population. Reports still display the unadjusted coverage definition above.

- Freshness: 95 percent of connected-device summaries appear within five minutes under the agreed load test. Offline periods are measured separately.
- Usability: a team leader can identify a data gap, compare an imported output measure, and resolve an exception without developer assistance in a scripted pilot session.
- Endpoint resource budget: provisional p95 collector CPU below 1 percent over sampled one-minute windows and resident memory below 150 MB on the nominated minimum-spec Windows machine during an eight-hour script. Collect baseline and added load, startup behavior, and VPN and endpoint-security effects. Revise targets only with measured evidence.
- Performance: p95 daily-team and 30-day aggregate views below two seconds at 20 simultaneous dashboard sessions and the 1,000-agent synthetic workload on a documented test host. Large exports run as bounded jobs.
- Security: tests deny access across team and client scopes, including exports and revoked users or devices. Sensitive screen text must be absent from payloads, local queues, logs, and backups.
- Recovery: demonstrate database restoration within four hours and no more than 24 hours of acknowledged-data loss with the initial daily-backup design. If the company needs tighter recovery, add and cost a tested continuous-archive design before rollout.

9 Collection and trust policy

Collect the minimum required to explain coarse workstation state: pseudonymous employee and device IDs, approved process identifiers, coarse state durations, sequence and version fields, and health diagnostics. Resolve employee names on the server only for authorized viewers.

Do not collect window titles, URLs, document names, typed characters, keystroke counts, screenshots, audio, clipboard contents, or mouse trails. No per-keystroke hooks are required for the first release. An idle threshold of five minutes is a configurable pilot default; changing it creates a new policy version. The exact threshold boundary semantics are defined in the technical plan.

Collection follows a company-managed work session and visible policy. Outside an explicit collection window the agent stops activity capture and reports only non-content health as authorized. A missing schedule does not silently enable all-day tracking. Device enrollment initially receives a test collection window; production windows derive from approved schedules and an explicit grace period, default zero. Expired offline policy stops collection and records a gap.

Proposed retention is 30 days for raw summaries and successful input source files, 13 months for daily aggregates, 90 days for audit and health logs, and 30 days for backup copies. Local queue retention is at most seven days or 100 MB, whichever limit is reached first. Rejected import files are deleted after seven days unless needed for an active correction. The company data owner validates these defaults before employee deployment. Deletion must cover primary data, derived views, source uploads, and eventual backup expiry; restoration reapplies a deletion ledger before access is restored. Retain minimal normalized schedules, operational facts, approved-hours facts, classifications, and correction or annotation history for 13 months to explain retained daily results. Keep import metadata for 13 months, with rejected-row payloads removed after seven days. Delete generated exports after 24 hours, independently of link expiry. Retain pseudonymous ingestion receipts for 37 days from acceptance. Retain the minimal deletion ledger until all affected backups have expired and restoration controls have been verified. A 30-day raw limit applies to the live database; backup rotation can retain a deleted record for up to 30 additional days. Operational logs expire after 90 days; approval history necessary to explain a retained correction follows the 13-month record lifetime.

10 Decision ownership and boundaries

The coordinating AI lead owns requirements, technical decisions, task sequencing, integration review, and the delivery report. Specialist agents work within bounded assignments and report evidence to the lead. The user is not asked to select routine implementation details.

The company retains authority over employee deployment, operational definitions, access to internal systems, expenditure, and operational policy. The lead bundles these concrete dependencies when they become necessary. Repository creation is optional until collaboration requires a remote; local planning and synthetic-data implementation can proceed without it.

No cloud resource, subscription, paid API, or employee rollout is created by this planning work. A production launch requires both a passed engineering gate and a company-designated operational owner who can receive and act on incidents.

PART 2

Technical Implementation Plan

Version 1.0 | 12 September 2026 | Status: implementation baseline pending environment evidence

1 Architecture decision

Use a modular monolith: a C# Windows session collector, an ASP.NET Core API and background worker, a React and TypeScript web interface, and PostgreSQL. Start with .NET 10 LTS and a supported PostgreSQL major selected against the company's deployment image. Pin exact package and image versions during setup and maintain security updates. .NET support status is documented by Microsoft [S2]. PostgreSQL uses a permissive open-source license [S1]. Neither fact eliminates compute, support, or maintenance costs.

The browser communicates with the API over HTTPS. The collector sends bounded summary batches over HTTPS to the same service. The API commits accepted records to PostgreSQL before acknowledging them. A worker claims database jobs and builds daily aggregates; the dashboard queries those aggregates. The initial worker can run in the same deployable application with separate execution and database connection limits. Split its process only when operational evidence requires it.

The Linux deployment contains the application, PostgreSQL, and a TLS reverse proxy or existing company ingress. Use a simple service or container deployment compatible with company operations. There is no Kubernetes, Kafka, Redis, ClickHouse, separate data warehouse, or external LLM in the first release. Use an existing company-controlled backup destination on a separate failure domain.

Desktop session helper -> HTTPS API -> PostgreSQL raw summaries -> aggregation worker -> daily read models -> management web interface.

Approved CSV exports -> validated import staging -> revisioned operational facts -> the same read models.

2 Deployment and identity

The first infrastructure candidate is an existing Linux VM with 4 vCPU, 16 GB RAM, and 200 GB SSD, plus independent backup capacity. This is a benchmark starting point, not a sizing guarantee. Reserve capacity for operating system, PostgreSQL WAL, indexes, imports, temporary query work, logs, backups in transit, and growth. Do not place the only backup on this VM.

Use the existing company OIDC identity provider when available. The web application uses server-managed secure, HttpOnly session cookies with CSRF protection for writes. Authorization is evaluated against database scopes, not merely identity-provider roles. If no suitable provider exists, the lead designs a bounded local-identity fallback and reviews its password reset and MFA requirements before the employee pilot; development can use seeded synthetic identities.

Device authentication is separate from manager login. A short-lived single-use enrollment token binds an approved device, immutable Windows user SID, and company employee in an enrollment. A physical computer may hold several user enrollments. The server issues a per-enrollment credential protected to the enrolled Windows user; support rotation, expiry, and revocation. The server derives enrollment, employee, and device identity from that credential and rejects spoofed body identifiers. Every event and receipt carries enrollment_id. Rotation preserves enrollment identity; reassignment creates a new enrollment and leaves historical ownership unchanged. Shared-device and fast-user-switch behavior must pass the Windows gate before those modes are enabled. No shared fleet secret is embedded in the binary.

Prefer existing VPN access or an approved company gateway. Do not expose PostgreSQL publicly. Configure HTTPS, firewall limits, request size limits, separate migration and runtime database roles, and

least-privilege backup access. SSO logout, account disabling, and device revocation must have explicit propagation tests.

3 Windows collection design

Run the collector in the interactive user session. A Windows service alone is not assumed to see user-session foreground activity. Use a minimal session helper; add a privileged installation or update service only if company deployment tooling cannot provide the required capability. Never add a kernel driver or intrusive process-protection scheme.

Use supported Windows interfaces for foreground-process and last-input observations; evaluate session-specific behavior on real Windows hardware and RDP or VDI if present [S3]. Resolve a normalized executable basename or allowlisted application identifier, never full paths containing personal data or window titles. Poll coarse state at an initial five-second cadence and observe lock, sleep, resume, logon, and logoff transitions. Document that brief application changes between samples may be missed.

Accumulate measured state durations into UTC-aligned 60-second buckets. Each bucket has one stable envelope with time-positioned application and state slices. Each slice carries `start_offset_ms` and `end_offset_ms` relative to the bucket UTC start, `application_id`, and state. Offsets are ordered, nonoverlapping, and bounded by the bucket. Preserve unobserved gaps; the sum of slices cannot exceed measured observed time. Treat timing as precise to the documented sampling resolution, not as exact reconstruction of unseen transitions. Collection cadence, bucket duration, and upload interval are different settings. Upload once per minute with random jitter; do not upload individual input events.

For the five-minute idle threshold, mark inactive from the point the elapsed time since last input reaches five minutes until new input or another state transition. Do not retroactively relabel the preceding five minutes. Screen lock is inactive immediately. Sleep and collector absence produce missing evidence rather than fabricated idle time. Cap final partial buckets at the last healthy sample; never fill a multi-hour sleep interval from one observation.

Use a monotonic clock for durations and UTC wall time for alignment. Include boot ID, session ID, sequence, client timestamp, received timestamp, and detected clock discontinuities. A clock jump or uncertain interval is flagged or quarantined; NTP is not used as proof that a client is truthful. For concurrent sessions, server reconciliation detects overlap; it does not sum durations beyond elapsed time or silently choose the most active device.

Application classification runs on the server using an effective-dated rule set and its ID is attached to derived results. The same application can serve different business roles. Browser activity remains generic in the first release; website-specific reporting requires a separately scoped later integration.

4 Local durability and transfer

Persist each completed bucket before transmission. SQLite holds encrypted payload blobs; use an authenticated-encryption key protected by Windows DPAPI in the collector's approved user context [S4]. Standard SQLite is not assumed to encrypt its database by itself. Keep content-free metadata minimal and verify that plaintext does not leak into WAL, temporary files, crash output, or diagnostic logs. Key lifecycle and recovery behavior are tested during agent upgrades and OS-profile changes.

Use a random event ID plus enrollment, device, boot, session, `collector_instance_id`, and increasing sequence to identify a bucket. Generate a new `collector_instance_id` for each collector process start; retries retain their original instance and sequence. Retain the exact serialized event on retry. A request contains schema version, batch ID, agent version, policy version, and bounded events. Start with at most 200 envelopes and 512 KB uncompressed per request; reject decompression bombs and oversized slice arrays. For application churn, cap slices at 60 per bucket. On reaching the cap, close detailed collection at the current offset and mark the remaining contiguous suffix as detail-unavailable, preserving its start and end

offsets. Do not claim active, inactive, or application detail for that suffix or improve coverage with it; it remains unknown in reconciliation. Flag the event as coarsened so capacity pressure is visible.

The API validates identity, allowed policy, durations, timestamp bounds, enum values, version compatibility, and batch limits. Return per-event accepted, already accepted, or rejected outcomes, only after transaction commit. Transient failure receives exponential backoff with jitter; an acknowledgment timeout is retried safely. Permanent rejection is visible as a data-quality issue and cannot cause endless retransmission.

Local pending storage is bounded by seven days and 100 MB. On cap exhaustion, remove the oldest pending payloads according to the recorded policy, increment a loss counter, and retain a compact missing-range manifest for upload. The agent UI and fleet view show the loss. Do not silently discard data or stop enforcing the cap. A queue wipe after lost keys is similarly explicit. Acknowledged payloads are purged locally promptly.

Determine age from immutable event occurrence end, not retry or receipt time. Accept ordinary late data up to seven days old; reject older automatic telemetry with a recorded reason. Quarantine unreasonable future timestamps, initially more than five minutes beyond received time. Quarantined events do not create accepted time totals. Late accepted events mark affected employee-days dirty for recomputation and increment the report revision.

5 Database model

Use one company deployment with explicit authorization scope on every business record. Avoid adding SaaS tenancy complexity while retaining stable company and client-account keys where appropriate.

- **employees:** stable company employee ID, display metadata, identity-provider subject, status, effective employment dates. Do not use email as a durable key.
- **teams, client_accounts, assignments:** effective-dated employee, role, team, and client scope. Reject conflicting primary assignments or represent intentional multi-account allocation explicitly.
- **access_grants:** principal, scope, permissions, effective dates. Keep platform administration separate from analytics access.
- **devices and enrollments:** device ID, enrollment ID, effective employee mapping, immutable OS user SID, credential version, approved status, last health, agent version, and revoked timestamp.
- **schedules and schedule_intervals:** external source ID, employee, start and end UTC, source timezone, expected state, break or absence type, import version, and conflict status.
- **collection_policies and application_rules:** effective range, authorized collection windows, idle threshold, retention settings, rule priority, matching application ID, role scope, and policy version.
- **activity_envelopes:** enrollment ID, device ID, event ID, employee ID, boot ID, session ID, collector instance ID, sequence, bucket start and end, observed duration, received time, schema and policy versions, payload checksum, quality flags, and bounded state slices.
- **ingestion_receipts:** unique enrollment and event ID, plus a second unique enrollment, collector instance ID, and sequence key; first accepted date, checksum, and receipt expiry. Identity or checksum conflicts reject the payload. Keep at least 37 days from first acceptance so retries within the seven-day late window cannot duplicate retained 30-day data.
- **import_batches and import_errors:** source type, checksum, mapping version, uploader, timestamps, source timezone, validation totals, errors, commit status, and superseded version.
- **work_items or work_daily_facts:** source system, work type, stable external item ID or declared daily aggregation key, employee mapping, completion date, count, handling duration when supplied, reviewed and passed counts, and import revision.

- `approved_hours_facts`: employee, source period, approved duration, external key, source timezone, import revision, and approval provenance; this optional feed is required for output-per-hour denominators.
- `daily_metrics`: employee, report date and timezone, metric version, rule version, source revisions, category durations, coverage, output and quality facts, conflict count, computed time, and report revision.
- `exceptions`, `annotations`, `correction_requests`: employee interval, source reference, author, reason, assigned owner, state, approved interpretation, and resolution history.
- `audit_events`, `device_health`, `job_queue`, `deletion_ledger`: scoped immutable history, operational diagnostics, durable work claims, and deletion requests that must be reapplied after restoration.

Store activity envelopes in daily UTC partitions with a primary key including the partition date. A nonpartitioned receipt table enforces event and collector-sequence deduplication across partition boundaries in the same transaction as accepted events and dirty-job updates. Expired replay is rejected by occurrence age even after a receipt expires. PostgreSQL partitioned uniqueness restrictions must be respected [S5]; do not assume a unique event ID alone can be declared globally on a date-partitioned table. If an existing event ID arrives with a changed checksum, reject it as a conflict instead of replacing the observation.

Index raw activity by employee and bucket start, receipts by expiry, summaries by team/date through effective assignments, jobs by ready state, and source records by external identity and revision. Retention drops or clears expired partitions only after related report and deletion checks. Raw data is for drill-down and recomputation within retention; standard dashboards use aggregates. Schema migrations and downgrade compatibility are reviewed explicitly.

6 Aggregation and metric implementation

The worker claims employee-day jobs using a database transaction, bounded batches, and a retry count. Jobs are deduplicated by employee, report date, timezone, and metric version. Ingress increments a `dirty_generation` counter in the same transaction as source changes. The worker captures that generation and a consistent input snapshot, computes a complete replacement aggregate, then publishes with a generation check. A stale worker cannot overwrite a newer published generation. Clear the job only if its current dirty generation equals the computed generation; otherwise keep it queued. This prevents arrivals during computation from being lost. Never increment totals blindly on every retry.

Split intervals at midnight in the report timezone, schedule boundaries, policy changes, and annotation boundaries. Reconcile overlaps using the project definition's precedence. Preserve distinct unknown, conflict, annotated, active, and locked or inactive durations. Also compute telemetry coverage independently from the raw union of valid intervals. Test daylight-saving transitions even if the pilot timezone does not use them.

Daily aggregates are authoritative only for their declared definitions and source revisions. Historical rule changes apply prospectively by default. A requested historical recalculation is possible only while required source detail is retained; old summaries are not presented as if rebuilt when the raw evidence has expired. Reports after retention still expose their metric version and whether reprocessing is possible.

CSV imports use staging and a preview before transactional commit. Reject ambiguous employee mappings, unsupported time formats, impossible counts, duplicate keys, and overlapping schedule periods. A corrected file supersedes a declared source period and revision atomically; an identical file is a no-op. Do not use fuzzy name matching. Work-item-level and daily-aggregate feeds cannot both contribute to the same measure for the same scope and period.

7 API and interface contracts

Initial versioned routes are POST /api/v1/enrollments/redeem, POST /api/v1/activity/batches, GET /api/v1/device/policy, POST /api/v1/device/health, POST /api/v1/imports/preview, POST /api/v1/imports/{id}/commit, GET /api/v1/teams/{id}/daily, GET /api/v1/employees/{id}/timeline, POST /api/v1/corrections, POST /api/v1/exceptions, and POST /api/v1/exports. Enrollment-token creation and policy administration are separately privileged actions.

Define OpenAPI schemas, error codes, pagination, time conventions, and size limits before parallel implementation. Return coverage, source freshness, report revision, units, and definition version with each analytics response. Missing optional operational data returns an availability reason, not a numeric zero. Test object-level access checks even when the requester guesses a valid ID.

The web interface contains team overview, employee explanation, output and quality, exceptions, imports, and device health. Lead with source quality. Show meaningful empty and stale states. Export scopes mirror view permissions, expire download links, and audit generation and retrieval. Neutralize formula-like CSV values without altering stored source records.

8 Capacity model and cost controls

The initial model assumes eight tracked hours per day, 60-second envelopes, 30 retained calendar days, and continuous activity on each modeled day for a conservative example. Envelope count per agent-day is $8 \times 60 = 480$. At 50 agents this is 24,000 per day; at 200 it is 96,000; at 1,000 it is 480,000. Thirty-day counts are 0.72 million, 2.88 million, and 14.4 million respectively.

At an illustrative 1 to 2 KB per stored envelope including bounded slices, 200 agents produce approximately 2.9 to 5.8 GB of raw retained payload; 1,000 produce 14.4 to 28.8 GB. These are decimal payload estimates, excluding indexes, receipts, WAL, backups, free space, and database overhead. The 1 to 2 KB assumption represents typical payloads, not a bound. Measure median and p95 bytes per envelope and application-switch distribution in the prototype; a 60-slice payload may be much larger. At 1,000 connected agents, one request per minute averages about 16.7 requests per second; reconnect storms and multi-envelope replays drive the more important burst test.

Maintain at least 30 percent free disk headroom as a proposed alert threshold. Track ingestion rate, queue age, worker lag, rows and bytes per agent-day, backup size, restore time, and p95 query latency. Benchmark 200 and 1,000 agents, a four-hour replay with a global admission limit, 20 concurrent report users, and a concurrent backup. Also demonstrate a full seven-day capped offline queue draining without gaps beyond documented eviction, and record its drain time and current-data freshness. Protect current ingestion from replay with per-device limits and a bounded shared replay budget.

Software license cost is not total cost. The ledger includes host allocation or rental, disks, backup storage, network transfer, Windows testing and CI, trusted code signing or existing internal PKI, identity costs if any, operator time, security patching, and AI development usage. No dollar amount is presented as a supplier quote.

Default incremental recurring software-service commitment is zero. Reuse facilities only after confirming entitlement and capacity. If an external expense becomes necessary, the lead presents a concrete option, actual current quote, alternative, and monthly or annual effect before purchase. Existing AI account limits are a project constraint; agents are bounded and reused rather than run continuously.

Consider a separate analytics engine only after indexes, partitioning, rollups, retention, query limits, and measured host sizing fail the agreed workload target. A sustained failure across repeated representative benchmarks, with a documented comparison of operator effort and infrastructure cost, is the trigger for an architecture decision. Self-hosted ClickHouse also adds operating cost.

9 Operations and release safety

Expose health endpoints and record content-free structured logs with request and event IDs. Alert through existing company channels on ingestion unavailability, oldest pending job, low disk, failed backup, expired signing material, and device version drift. Synthetic data is used in development and CI; production data does not enter AI prompts or public issue trackers.

Start with daily encrypted off-host database backups, 30-day rotation, documented secret recovery, and a restore rehearsal before pilot. Target RPO 24 hours and RTO four hours. Acknowledged events purged from endpoints cannot be assumed recoverable after server backup loss. Tighter RPO requires a separately tested and costed WAL archive or equivalent backup design [S6].

Roll out an agent first to controlled test devices, then five pilot canaries, then 25 to 50 pilot users, and only then to 200. Use company-approved signed artifacts and endpoint deployment tooling. Code signing does not guarantee antivirus acceptance; test actual company security software. Updates verify authenticity, stage safely, preserve compatible queued data, and retain the last working version. Maintain server compatibility with at least the current and previous agent schema during rollout.

For incidents, the operations owner can disable collection policy, revoke credentials, stop updates, restrict reports, and restore a known release. Critical database migrations use an expand-and-contract approach with a tested backup and rollback or forward-fix plan. Retention tasks have dry-run counts and bounded permissions. Implement every lifetime in project definition section 9, including normalized facts, correction history, receipts, exports, and the deletion ledger; link expiry alone does not delete an export. Do not remove historical grants or classification versions while retained results still need them for explanation or access control. Export downloads and incident records respect the same employee-data access boundaries.

10 Delivery work packages and evidence gates

Timing is an initial engineering planning range, not a six-month promise. A working pilot is estimated at 8 to 12 active engineering weeks after environment access, based on one coordinating lead and up to three specialist agents. Windows availability, review capacity, company source quality, and operational rollout can extend elapsed time. Re-estimate after the first end-to-end slice. No claim is made that an AI agent week equals a full human engineering week.

G0 Foundation and environment evidence, indicative week 1: establish repository structure, decisions, OpenAPI and sample contracts, synthetic fixtures, host and Windows inventory, metric tests, and dependency license record. Exit evidence is a runnable development environment and an explicit test matrix. Unknown identity or deployment details remain tracked dependencies.

G1 End-to-end measurement spike, indicative weeks 2 to 3: collect approved coarse state on a controlled Windows test device, encrypt and replay a local queue, ingest into PostgreSQL, and display one employee-day. Exit evidence includes restart and duplicate replay tests, lock and sleep behavior, plaintext inspection, and measured resource usage. Failure here changes the approach before dashboard expansion.

G2 Operational model and access, indicative weeks 4 to 5: implement roster, schedule, one output feed, scoped identity, rules, and revisioned aggregates. Exit evidence includes source reconciliation, overnight and DST tests, no-double-count invariants, and denied cross-scope access. Access gates must pass before real employee records are loaded.

G3 Management workflow and reliability, indicative weeks 6 to 8: deliver overview, timelines, corrections, exceptions, exports, device health, retention, updates, and backup procedures. Exit evidence includes manager task scripts, 200 and 1,000-agent benchmarks, reconnect admission control, a restore rehearsal, and independent review of security-sensitive changes.

G4 Controlled pilot and hardening, indicative weeks 9 to 12 with at least two weeks of observation: canary and 25 to 50-user rollout, comparison with authoritative source totals, report-preparation baseline comparison, and issue correction. Exit requires the project definition's quality gates, documented business findings, resolved critical defects, and a named company operations owner. The lead records pass, fail, or conditional status with evidence for every gate.

G5 Expansion to 200 and handover follows G4: verify capacity on the actual host, complete runbooks, rehearse agent rollback, and review cost per active device. Expansion beyond 200 is a new measured gate, not implied by synthetic 1,000-agent testing.

11 Required verification matrix

- Domain: no duplicate time; exact category reconciliation; zero and unavailable distinction; missing schedule; overnight shifts; DST; idle boundary; conflicting devices; effective team changes; annotations and report revisions.
- Ingestion: timeout after commit; duplicate or changed payload; out-of-order sequence; reconnect storm; malformed version; revoked device; clock skew; queue cap; lost encryption key; expired policy; seven-day replay boundary.
- Imports: wrong timezone; missing employee ID; repeated upload; partial errors; corrected source period; quality numerator beyond denominator; concurrent source versions; CSV export formula safety.
- Authorization: employee self-view; team and client denial; historical transfer; report-job execution and download scope; admin without analytics grant; revoked identity session.
- Windows: supported minimum hardware; install and uninstall; lock and unlock; sleep; reboot; fast-user switching; VPN; RDP or VDI if in scope; endpoint-security compatibility; update and rollback; offline policy expiry.
- Operations: backup failure alert; actual restore; deletion ledger replay; retention expiry; low disk; exhausted connections; worker crash mid-job; schema rollback compatibility; representative load under backup.

Automated tests run in CI where possible. Real Windows behavior, deployment policy, and user workflows require recorded environment evidence. A simulation passing on macOS or Linux does not close the Windows gate.

12 Technical references

References checked 12 September 2026. They support platform behavior and licensing, not company-specific feasibility or pricing. Revalidate supported versions at implementation.

[S1] PostgreSQL License. <https://www.postgresql.org/about/licence/>

[S2] Microsoft .NET support policy. <https://dotnet.microsoft.com/en-us/platform/support/policy>

[S3] Microsoft GetLastInputInfo function.

<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getlastinputinfo>

[S4] Microsoft CryptProtectData function.

<https://learn.microsoft.com/en-us/windows/win32/api/dpapi/nf-dpapi-cryptprotectdata>

[S5] PostgreSQL table partitioning and unique constraints. <https://www.postgresql.org/docs/current/ddl-partitioning.html>

[S6] PostgreSQL backup and restore. <https://www.postgresql.org/docs/current/backup.html>

PART 3

AI Team Delivery and Project Control

Version 1.0 | 12 September 2026

1 Operating model

The coordinating AI lead is the project manager and integration owner. The lead chooses the next work package, delegates bounded tasks, reviews evidence, resolves conflicts, and maintains one accepted baseline. The user receives outcomes and indispensable dependency requests, rather than a menu of routine decisions.

This model has been used for the planning work: product, architecture, and delivery agents produced independent contributions; the lead read and consolidated them, then requested a second review of the baseline. Those contributions remain in docs/reviews for traceability. They are advisory, and their alternative metrics, recovery targets, or milestone labels do not override the three numbered baseline documents.

Agents execute while assigned work is active. This document does not establish an unattended scheduler or promise work will continue after an active run ends. Future work resumes from the saved backlog and decision log, rather than relying on an agent remembering the conversation.

2 Roles and assignment rules

The lead owns product scope, dependency choices, budget recommendations, task sequencing, shared contracts, integration tests, and release recommendations. The lead also performs local work while specialist tasks run.

The product and data role owns metric examples, import mappings, workflows, requirements traceability, and pilot evaluation. The platform and data role owns API, PostgreSQL, ingestion, aggregation, identity, and operations. The endpoint role owns the Windows collector, local queue, installer, and real-device evidence. The interface role owns manager and employee views. The independent quality and security role challenges assumptions, tests boundaries, and reviews failure cases.

These are rotating responsibilities, not six permanently running agents. Use at most the available four concurrent slots: one lead and up to three specialists. Activate only roles with independent, useful work. A contributor cannot be the sole reviewer of its own change; another agent reviews it before the lead accepts it. Security-sensitive collection, authorization, update, and deletion changes receive explicit independent review.

Before delegation, the lead assigns a work-package ID, objective, dependency state, permitted files, prohibited scope, acceptance criteria, required tests, and report format. Shared contracts are finalized first. Agents do not modify a shared schema or API contract without returning the proposed change to the lead. Once Git is available, isolated worktrees are preferred for overlapping development; otherwise use exclusive file ownership.

3 Required agent report

Each report contains the package ID, changed file paths, decisions and assumptions, acceptance evidence, commands actually run and results, checks not run and why, open findings with severity, cost or scope impact, and the recommended next step. A statement that work is complete is not test evidence.

The lead checks the actual diff or artifact, relevant test output, and cross-module behavior. Package outcomes are accepted, revision required, or blocked on a named dependency. Critical data loss, exposure, or incorrect totals prevent acceptance. Lower-priority findings may be deferred only with an owner, reason, and target gate. Document acceptance never implies software tests passed.

4 Initial implementation queue

- WP01 Contracts and fixtures. Owner: product and data. Define CSV templates, OpenAPI shapes, state transitions, employee-day reference fixtures, and source-version rules. Reviewer: platform. Depends on the accepted planning baseline. Exit: independently calculated expected totals for at least eight edge-case days and valid and invalid examples for each import.
- WP02 Development foundation. Owner: platform. Create the .NET and web skeleton, PostgreSQL migrations, synthetic identity, repeatable local startup, and meaningful CI checks. Reviewer: quality. Depends on WP01 contract boundaries. Exit: clean setup from instructions, health check, scoped synthetic request, migration test, and no required paid account.
- WP03 Windows feasibility. Owner: endpoint. Implement the bounded collector state machine and encrypted queue on a controlled test device. Reviewer: platform and quality. Can run alongside WP02 after WP01. Exit: observed Windows state transitions, replay, resource measurements, and content inspection. Windows access is a real dependency for this exit.
- WP04 Durable ingestion and aggregation. Owner: platform. Connect device enrollment, idempotent receipts, versioned daily aggregation, and queue jobs. Reviewer: endpoint and quality. Depends on WP02 and the agreed WP03 event contract. Exit: retry, late-arrival, overlap, clock, and transaction-crash fixtures pass.
- WP05 Sources and identity. Owner: product and platform in separate files. Add company login integration, scope enforcement, roster, schedule, and the first operational feed. Reviewer: quality. Depends on WP02 and WP01. Exit: source totals reconcile, repeat imports are safe, and guessed cross-scope IDs are denied.
- WP06 Decision workflows. Owner: interface. Add overview, employee explanation, corrections, exceptions, and safe exports against the approved API. Reviewer: product and quality. Depends on WP04 and WP05; mocked interface work may begin sooner. Exit: manager task scripts pass with missing, conflicting, and corrected evidence.
- WP07 Operational readiness. Owner: platform and endpoint in separate packages. Add policy rollout, signed update path, retention, backup, alerts, runbooks, and bounded load tests. Reviewer: quality. Depends on WP03 through WP06. Exit: measured restore, rollback, access, and 200 and 1,000-agent test reports.
- WP08 Pilot and release review. Owner: lead with company operations delegate. Gather baseline and at least two weeks of pilot evidence; resolve defects and record rollout decision. Depends on WP07 and company employee-deployment authorization. Exit: every G4 gate has evidence and the operational owner accepts support responsibility.

The lead starts with WP01 and WP02 when implementation begins; Windows feasibility is brought forward as soon as a controlled device is available. Repository creation is not a prerequisite for improving the local contracts. These work packages are queued, not reported as implemented.

5 Cost and change control

The default is no new recurring software-service commitment. Agents may select open-source libraries within the approved stack after license and maintenance review. A new service, expanded collection field, runtime AI call, additional platform, or larger rollout requires a recorded decision explaining value, alternatives, tests, operating effort, and cost.

Maintain a cost ledger with quantity, unit, current allocation, incremental cost, price source and date, payer, spending authority, and renewal or capacity limit. Unverified costs remain unquoted rather than being entered as zero. Initial entries are:

- Company Linux VM and disk: proposed reuse; availability and internal allocation unknown; no purchase committed.
- Independent backup destination: proposed reuse; capacity and restore access unknown; no purchase committed.
- Company identity and endpoint management: existing entitlement to be checked; do not assume additional features are included.
- Windows test machine and build runner: access pending; existing-device route preferred.
- Signing and installer distribution: company PKI or signing facility to be checked; any certificate purchase remains a separate budget decision.
- Development AI usage: uses the current account; bounded delegation consumes its available usage; no new API subscription is planned.
- Operations time and security maintenance: identify the company owner and recurring effort during pilot; it is part of total cost even without a vendor bill.

Monthly incremental cost equals incremental compute plus storage plus backup plus transfer plus signing allocation plus build and identity charges plus AI charges, where applicable. Also report internally allocated facilities and operator hours separately. A paid proposal must include an actual quote and a lower-cost alternative before expenditure.

6 Risk register and ownership

- K01 Misleading performance interpretation. Owner: product. Mitigation: separate activity, coverage, context, and output; require source and denominator labels. Release blocker: a view presents missing data as zero work or activity as billable output.
- K02 Windows or VDI incompatibility. Owner: endpoint. Mitigation: real-device spike before interface expansion. Trigger: state or permission behavior differs from assumptions; narrow supported scope and re-estimate.
- K03 Lost or duplicated intervals. Owner: platform. Mitigation: local durability, global receipts, deterministic recomputation, overlap and replay tests. Release blocker: unexplained category totals or acknowledged loss outside the declared recovery model.
- K04 Unauthorized employee detail. Owner: quality and security. Mitigation: scoped API and export tests, revocation, minimum collection. Release blocker: any unresolved unauthorized-access finding.
- K05 Hidden operating cost. Owner: lead. Mitigation: allocation ledger, measured storage, no automatic service signup. Trigger: any proposal adds a recurring dependency or exceeds measured host capacity.
- K06 Unrecoverable server. Owner: operations. Mitigation: independent backups, key recovery, restore drills. Release blocker: restoration has not been demonstrated against the target.
- K07 Weak operational sources. Owner: product and company data owner. Mitigation: one validated feed, exact identity mapping, revisioned source totals. Trigger: no authoritative output feed; label time-visibility-only and hold the output-linkage value gate.
- K08 AI work accepted without evidence. Owner: lead. Mitigation: independent review, actual checks, fixed ownership, saved decisions. Release blocker: a required gate is asserted from simulation or narrative alone.
- K09 No accountable company operator. Owner: lead and company sponsor. Mitigation: name the incident and data-policy owners before pilot. Continue synthetic work while access is pending.

7 Reporting and company dependencies

At a meaningful milestone, report what is accepted, what was actually verified, the next work package, a material forecast or cost change, and any indispensable company action. Do not ask the user to choose which ordinary task happens next.

Bundle environment dependencies into a concrete request when needed: repository destination if remote collaboration is required; access to a controlled Windows device; approved anonymized source exports; company identity and hosting details; and the appropriate operations or IT delegate. Credentials belong in secure configuration, never chat, documents, commits, or agent reports.

Engineering autonomy does not substitute for company authority over employees, secrets, or spending. The lead prepares a reviewable configuration, deployment package, collection notice, and test record before asking for the final company deployment action. Repository creation and granting access may be performed by the user or their delegate; routine implementation remains the lead's responsibility.